

Python Encapsulation: Interview Practice

30 technical practice questions — Public, Protected, and Private variables

Goal: Solve these yourself first. The questions move from easy to advanced and include debugging, reasoning, inheritance, and interview-style challenges.

1. Public Variables — 10 Questions

Easy 1: Create a class **Student** with a public variable **name**. Create an object and print the name directly using the object.

Easy 2: Create a class **Car** with a public variable **color**. Change the color through the object and print the new value.

Easy 3: Create a class **Book** with public variables **title** and **price**. Create an object and print both values.

Easy 4: Create a class **Employee** with a public variable **salary**. Increase the salary directly through the object and print the result.

Medium 5: Create a class **Student** with public variables **name** and **marks**. Write a method that prints the student's name and average mark.

Medium 6: Create a class **BankAccount** with a public variable **balance**. Write methods for deposit and withdrawal. Then prove that the balance can also be changed directly from outside the class.

Medium 7: Create a class **Product** with public variables **name** and **stock**. Write a method that decreases stock after a sale. What happens if outside code sets stock to -10? Handle the situation.

Hard 8: Two objects of a class have a public variable **score**. Change the score of only the first object and print both scores. Explain why the second object does not change.

Hard 9: Create a class **Temperature** with a public variable **celsius**. Write a method to convert it to Fahrenheit. Then change celsius directly outside the class and call the method again. What does this demonstrate about public attributes?

Advanced 10: Interview challenge: A class stores a public variable **age**. Another programmer sets **age = -50** from outside the class. Explain the design problem and rewrite the class using a more controlled approach, without using properties or decorators.

2. Protected Variables — 10 Questions

Easy 1: Create a class **Employee** with a protected variable `_salary`. Create an object and print the value. Explain what the single underscore means in Python.

Easy 2: Create a class **Person** with a protected variable `_name`. Create a method that prints it. Then access `_name` from outside and observe what happens.

Easy 3: Create a class **Vehicle** with a protected variable `_speed`. Create a subclass **Car** that reads `_speed` and prints it.

Easy 4: Create a class **Parent** with `_value = 100` and a child class **Child**. Access `_value` inside the child class. Does Python stop the access?

Medium 5: Create a class **BankAccount** with protected variable `_balance`. Create deposit and withdrawal methods. Try changing `_balance` directly outside the class and explain why it is still possible.

Medium 6: Create a parent class **Animal** with `_sound` and a child class **Dog** that uses `_sound`. Then create an object of **Dog** and inspect the variable from outside.

Medium 7: Create a class **Game** with protected variables `_score` and `_level`. Write methods to update them. In a subclass, create a method that displays both values.

Hard 8: Create a parent class **Account** with `_balance` and a child class **SavingsAccount**. The child should add interest using `_balance`. Explain why protected variables are useful in inheritance.

Hard 9: Interview challenge: A developer says, 'A protected variable cannot be accessed outside the class.' Is that statement correct in Python? Write a small program that proves your answer.

Advanced 10: Design a parent class **Employee** with `_salary` and a child class **Manager**. Allow the child to use `_salary`, but make your code clearly communicate that outside code should not access it directly. Explain the convention involved.

3. Private Variables — 10 Questions

Easy 1: Create a class **Student** with a private variable `__marks`. Create a method `show_marks()` that prints the marks.

Easy 2: Create a class **BankAccount** with a private variable `__balance`. Add deposit and withdrawal methods that modify the balance.

Easy 3: Create a class **User** with a private variable `__password`. Create a method `check_password()` that checks whether a given password matches.

Easy 4: Create a class **Employee** with private variable `__salary`. Create a method that returns the salary. Try accessing `__salary` directly from outside and record the error.

Medium 5: Create a class **Counter** with private variable `__count`. Add `increment()` and `show_count()` methods. Increase the count five times and print it.

Medium 6: Create a class **ATM** with private variable `__pin`. Write a method that checks a PIN. Do not print the actual PIN. Test both a correct and incorrect PIN.

Medium 7: Create a class **Student** with private variable `__marks` as a list. Add a method to calculate the average. Add another method that rejects marks below 0 or above 100.

Hard 8: Create a class **BankAccount** with private variable `__balance`. Add `deposit()`, `withdraw()`, and `show_balance()`. Prevent withdrawal when the requested amount is greater than the balance.

Hard 9: Interview challenge: Create an object containing `__data`. Try accessing `__data` directly and then inspect the object's attributes using `__dict__`. Explain what Python's name mangling is doing.

Advanced 10: Technical challenge: Create a class **Employee** with private `__salary`. Add a method `increase_salary(percent)` that validates the percentage and updates the private value. Then create a subclass and investigate whether it can directly access `__salary`. Explain the result and the reason.